Assignment 1

Task 4: Writing Stored Procedures

Group Member Names:

Madhavi Latha Tripuram - C3528532

Hari Krishna Jadda - C3511138

Dhimanshi Piripun Jayasena - C3503338

Lab Timings: **Thursday 1.00pm to 3.00pm**

Course coordinator: **Prof. Mahakim Newton**

Writing Stored Procedures

```
-- Stored Procedures for GlobalCarHire System
-- INFO6002 Database Management 2 - Assignment 1
-- Group Members:
-- Madhavi Latha Tripuram - C3528532
-- Hari Krishna Jadda - C3511138
-- Dhimanshi Piripun Jayasena - C3503338

USE GlobalCarHire;
GO

-- Procedure 1: Create a new booking
CREATE OR ALTER PROCEDURE sp_CreateBooking
    @bookingId VARCHAR(10),
    @modeOfBooking VARCHAR(10),
    @customerId VARCHAR(10),
    @serviceStartDate DATE,
```

```
    @serviceEndDate DATE,
    @paymentMethod VARCHAR(10),
    @bookingDeposit DECIMAL(10,2) = 0,
    @employeeId VARCHAR(10),
    @pricedServiceIds VARCHAR(MAX), -- Comma-separated list of pricedServiceIds
    @vehicleIds VARCHAR(MAX), -- Comma-separated list of vehicleIds
    @driverIds VARCHAR(MAX) = NULL, -- Comma-separated list of driverIds (optional)
    @extraServices VARCHAR(MAX) = NULL, -- Comma-separated list of extraServiceId:quantity pairs
    @result INT OUTPUT,
    @message VARCHAR(255) OUTPUT
AS
BEGIN
    SET NOCOUNT ON;
    BEGIN TRY
        BEGIN TRANSACTION;

        -- Validate input parameters
        IF @serviceEndDate < @serviceStartDate
        BEGIN
            SET @result = 0;
            SET @message = 'Service end date must be after start date.';
            ROLLBACK;
            RETURN;
        END

        -- Check if customer exists
        IF NOT EXISTS (SELECT 1 FROM Customer WHERE customerId = @customerId)
        BEGIN
```

```
        SET @result = 0;
        SET @message = 'Customer does not exist.';
        ROLLBACK;
        RETURN;
    END

    -- Check if employee exists and is active
    IF NOT EXISTS (SELECT 1 FROM Employee WHERE employeeId =
@employeeId)
    BEGIN
        SET @result = 0;
        SET @message = 'Employee does not exist.';
        ROLLBACK;
        RETURN;
    END

    -- Check if all vehicles are available for the requested period
    DECLARE @vehicleTable TABLE (vehicleId VARCHAR(10));
    INSERT INTO @vehicleTable SELECT value FROM
STRING_SPLIT(@vehicleIds, ',');

    DECLARE @unavailableVehicles INT;
    SELECT @unavailableVehicles = COUNT(*)
    FROM @vehicleTable v
    JOIN Vehicle veh ON v.vehicleId = veh.vehicleId
    WHERE veh.status != 'Active'
      OR EXISTS (
        SELECT 1
        FROM Booking_Vehicle bv
        JOIN Booking b ON bv.bookingId = b.bookingId
        WHERE bv.vehicleId = veh.vehicleId
```

```
            AND b.bookingStatus = 'Confirmed'
            AND (
                (@serviceStartDate BETWEEN b.serviceStartDate AND
b.serviceEndDate)
                OR (@serviceEndDate BETWEEN b.serviceStartDate AND
b.serviceEndDate)
                OR (b.serviceStartDate BETWEEN @serviceStartDate AND
@serviceEndDate)
                OR (b.serviceEndDate BETWEEN @serviceStartDate AND
@serviceEndDate)
            )
        );

    IF @unavailableVehicles > 0
    BEGIN
        SET @result = 0;
        SET @message = 'One or more vehicles are not available for the requested
dates.';
        ROLLBACK;
        RETURN;
    END

    -- Check if all priced services exist and are active
    DECLARE @pricedServiceTable TABLE (pricedServiceId VARCHAR(10));
    INSERT INTO @pricedServiceTable SELECT value FROM
STRING_SPLIT(@pricedServiceIds, ',');

    DECLARE @invalidPricedServices INT;
    SELECT @invalidPricedServices = COUNT(*)
    FROM @pricedServiceTable p
    LEFT JOIN PricedService ps ON p.pricedServiceId = ps.pricedServiceId
    LEFT JOIN Service s ON ps.serviceId = s.serviceId
```

```
    WHERE ps.pricedServiceId IS NULL OR s.status != 'Available';

    IF @invalidPricedServices > 0
    BEGIN
        SET @result = 0;
        SET @message = 'One or more priced services are invalid or unavailable.';
        ROLLBACK;
        RETURN;
    END

    -- Check if all drivers exist (if provided)
    IF @driverIds IS NOT NULL
    BEGIN
        DECLARE @driverTable TABLE (driverId VARCHAR(10));
        INSERT INTO @driverTable SELECT value FROM
STRING_SPLIT(@driverIds, ',');

        DECLARE @invalidDrivers INT;
        SELECT @invalidDrivers = COUNT(*)
        FROM @driverTable d
        LEFT JOIN Driver dr ON d.driverId = dr.driverId
        WHERE dr.driverId IS NULL;

        IF @invalidDrivers > 0
        BEGIN
            SET @result = 0;
            SET @message = 'One or more drivers do not exist.';
            ROLLBACK;
            RETURN;
        END
    END
```

```
    -- Check if all extra services exist (if provided)
    IF @extraServices IS NOT NULL
    BEGIN
        DECLARE @extraServiceTable TABLE (extraServiceId VARCHAR(10), quantity INT);

        -- Parse the extra services string (format: "id1:qty1,id2:qty2,...")
        DECLARE @pos INT, @nextPos INT, @item VARCHAR(50), @colonPos INT;
        SET @pos = 1;

        WHILE @pos <= LEN(@extraServices)
        BEGIN
            SET @nextPos = CHARINDEX(',', @extraServices, @pos);
            IF @nextPos = 0 SET @nextPos = LEN(@extraServices) + 1;
            SET @item = SUBSTRING(@extraServices, @pos, @nextPos - @pos);
            SET @colonPos = CHARINDEX(':', @item);

            IF @colonPos > 0
            BEGIN
                INSERT INTO @extraServiceTable VALUES (
                    SUBSTRING(@item, 1, @colonPos - 1),
                    CAST(SUBSTRING(@item, @colonPos + 1, LEN(@item) - @colonPos) AS INT)
                );
            END

            SET @pos = @nextPos + 1;
        END
```

```
        DECLARE @invalidExtraServices INT;
        SELECT @invalidExtraServices = COUNT(*)
        FROM @extraServiceTable e
        LEFT JOIN ExtraService es ON e.extraServiceId = es.extraServiceId
        WHERE es.extraServiceId IS NULL OR e.quantity <= 0;

        IF @invalidExtraServices > 0
        BEGIN
            SET @result = 0;
            SET @message = 'One or more extra services are invalid or have invalid quantities.';
            ROLLBACK;
            RETURN;
        END
    END

    -- Get customer information
    DECLARE @customerInfo VARCHAR(50);
    SELECT @customerInfo = fullName FROM Customer WHERE customerId = @customerId;

    -- Create the booking record
    INSERT INTO Booking (
        bookingId, modeOfBooking, customerInformation,
        numberOfPricedServices, numberOfVehicles,
        serviceStartDate, serviceEndDate, paymentMethod,
        bookingDeposit, bookingStatus, customerId
    ) VALUES (
        @bookingId, @modeOfBooking, @customerInfo,
        (SELECT COUNT(*) FROM @pricedServiceTable), (SELECT COUNT(*) FROM @vehicleTable),
```

```
    @serviceStartDate, @serviceEndDate, @paymentMethod,
    @bookingDeposit, 'Confirmed', @customerId
);

-- Add priced services to booking
INSERT INTO Booking_PricedService (bookingId, pricedServiceId)
SELECT @bookingId, pricedServiceId FROM @pricedServiceTable;

-- Add vehicles to booking
INSERT INTO Booking_Vehicle (bookingId, vehicleId)
SELECT @bookingId, vehicleId FROM @vehicleTable;

-- Add drivers to booking (if provided)
IF @driverIds IS NOT NULL
BEGIN
    INSERT INTO Booking_Driver (bookingId, driverId)
    SELECT @bookingId, driverId FROM @driverTable;
END

-- Add extra services to booking (if provided)
IF @extraServices IS NOT NULL
BEGIN
    INSERT INTO Booking_ExtraService (bookingId, extraServiceId, quantity)
    SELECT @bookingId, extraServiceId, quantity FROM @extraServiceTable;
END

-- Create initial pickup transaction
DECLARE @transactionId VARCHAR(10) = 'T' + RIGHT('0000000' +
CAST(ABS(CHECKSUM(NEWID())) % 10000000 AS VARCHAR(7));

INSERT INTO VehicleTransaction (
```

```
            transactionId, transactionType, actualDateTime,
            vehicleCondition, mileageReading, fuelLevel,
            additionalCharges, pickupDateTime, returnDateTime,
            pickupEmployeeId, returnEmployeeId, bookingId,
            vehicleId, employeeId
        )
        SELECT
            @transactionId, 'Pickup', GETDATE(),
            'Initial pickup - condition to be verified',
            v.mileage, 100, -- Assuming full tank at pickup
            0, @serviceStartDate, NULL,
            @employeeId, NULL, @bookingId,
            v.vehicleId, @employeeId
        FROM Vehicle v
        JOIN @vehicleTable vt ON v.vehicleId = vt.vehicleId;

        SET @result = 1;
        SET @message = 'Booking created successfully.';

        COMMIT;
    END TRY
    BEGIN CATCH
        IF @@TRANCOUNT > 0
            ROLLBACK;

        SET @result = 0;
        SET @message = 'Error creating booking: ' + ERROR_MESSAGE();
    END CATCH
END;
GO
```

```
-- Procedure 2: Process vehicle return
CREATE OR ALTER PROCEDURE sp_ProcessVehicleReturn
    @bookingId VARCHAR(10),
    @employeeId VARCHAR(10),
    @vehicleCondition TEXT,
    @mileageReading INT,
    @fuelLevel TINYINT,
    @damagePenalty DECIMAL(10,2) = 0,
    @latePenalty DECIMAL(10,2) = 0,
    @result INT OUTPUT,
    @message VARCHAR(255) OUTPUT,
    @invoiceId VARCHAR(10) OUTPUT
AS
BEGIN
    SET NOCOUNT ON;
    BEGIN TRY
        BEGIN TRANSACTION;


        -- Validate input parameters
        IF NOT EXISTS (SELECT 1 FROM Booking WHERE bookingId = @bookingId
AND bookingStatus = 'Confirmed')
        BEGIN
            SET @result = 0;
            SET @message = 'Booking does not exist or is not confirmed.';
            ROLLBACK;
            RETURN;
        END


        IF NOT EXISTS (SELECT 1 FROM Employee WHERE employeeId =
@employeeId)
```

```
BEGIN
    SET @result = 0;
    SET @message = 'Employee does not exist.';
    ROLLBACK;
    RETURN;
END

IF @fuelLevel < 0 OR @fuelLevel > 100
BEGIN
    SET @result = 0;
    SET @message = 'Fuel level must be between 0 and 100.';
    ROLLBACK;
    RETURN;
END

-- Get booking details
DECLARE @customerId VARCHAR(10), @serviceEndDate DATE;
SELECT @customerId = customerId, @serviceEndDate = serviceEndDate
FROM Booking
WHERE bookingId = @bookingId;

-- Check if return is late
DECLARE @isLate BIT = 0;
IF GETDATE() > @serviceEndDate
    SET @isLate = 1;

-- Update vehicle mileage
UPDATE v
SET v.mileage = @mileageReading
FROM Vehicle v
```

```
    JOIN Booking_Vehicle bv ON v.vehicleId = bv.vehicleId
    WHERE bv.bookingId = @bookingId;

    -- Update the pickup transaction with return details
    DECLARE @pickupTransactionId VARCHAR(10);
    SELECT @pickupTransactionId = transactionId
    FROM VehicleTransaction
    WHERE bookingId = @bookingId AND transactionType = 'Pickup';

    UPDATE VehicleTransaction
    SET
        returnDateTime = GETDATE(),
        returnEmployeeId = @employeeId,
        vehicleCondition = @vehicleCondition,
        mileageReading = @mileageReading,
        fuelLevel = @fuelLevel,
        additionalCharges = @damagePenalty + @latePenalty
    WHERE transactionId = @pickupTransactionId;

    -- Create return transaction
    DECLARE @returnTransactionId VARCHAR(10) = 'T' + RIGHT('0000000' +
CAST(ABS(CHECKSUM(NEWID())) % 10000000 AS VARCHAR(7)));

    INSERT INTO VehicleTransaction (
        transactionId, transactionType, actualDateTime,
        vehicleCondition, mileageReading, fuelLevel,
        additionalCharges, pickupDateTime, returnDateTime,
        pickupEmployeeId, returnEmployeeId, bookingId,
        vehicleId, employeeId
    )
    SELECT
```

```
        @returnTransactionId, 'Return', GETDATE(),
        @vehicleCondition, @mileageReading, @fuelLevel,
        @damagePenalty + @latePenalty,
        vt.pickupDateTime, GETDATE(),
        vt.pickupEmployeeId, @employeeId, @bookingId,
        vt.vehicleId, @employeeId
    FROM VehicleTransaction vt
    WHERE vt.transactionId = @pickupTransactionId;

    -- Calculate rental charges
    DECLARE @baseRentalCharges DECIMAL(10,2) = 0;
    DECLARE @extraServiceCharges DECIMAL(10,2) = 0;
    DECLARE @totalDiscount DECIMAL(10,2) = 0;
    DECLARE @loyaltyDiscountApplied BIT = 0;
    DECLARE @tierName VARCHAR(20) = NULL;

    -- Calculate base rental charges (sum of all priced services)
    SELECT @baseRentalCharges = SUM(ps.price *
        DATEDIFF(DAY,
            CASE WHEN b.serviceStartDate > ps.pricingStartDate THEN
b.serviceStartDate ELSE ps.pricingStartDate END,
            CASE WHEN b.serviceEndDate < ps.pricingEndDate THEN
b.serviceEndDate ELSE ps.pricingEndDate END
        ))
    FROM Booking_PricedService bps
    JOIN PricedService ps ON bps.pricedServiceId = ps.pricedServiceId
    JOIN Booking b ON bps.bookingId = b.bookingId
    WHERE bps.bookingId = @bookingId;

    -- Calculate extra service charges
    SELECT @extraServiceCharges = SUM(es.advertisedRate * bes.quantity)
```

```
    FROM Booking_ExtraService bes
    JOIN ExtraService es ON bes.extraServiceId = es.extraServiceId
    WHERE bes.bookingId = @bookingId;

    -- Apply loyalty discount if applicable
    SELECT
        @tierName = tierName,
        @totalDiscount = @baseRentalCharges * (ld.discountPercentage / 100),
        @loyaltyDiscountApplied = 1
    FROM Customer_LoyaltyDiscount cld
    JOIN LoyaltyDiscount ld ON cld.tierName = ld.tierName
    WHERE cld.customerId = @customerId;

    -- Create invoice
    SET @invoiceId = 'INV' + RIGHT('0000000' +
CAST(ABS(CHECKSUM(NEWID())) % 10000000 AS VARCHAR(7));

    INSERT INTO Invoice (
        invoiceId, baseRentalCharges, extraServiceCharges,
        totalLatePenalty, totalDamagePenalty, totalDiscount,
        finalAmount, paymentMethod, discountApprovedBy,
        loyaltyDiscountApplied, bookingId, employeeId, tierName
    ) VALUES (
        @invoiceId, @baseRentalCharges, @extraServiceCharges,
        @latePenalty, @damagePenalty, @totalDiscount,
        (@baseRentalCharges + @extraServiceCharges + @latePenalty +
@damagePenalty - @totalDiscount),
        (SELECT paymentMethod FROM Booking WHERE bookingId =
@bookingId),
        CASE WHEN @loyaltyDiscountApplied = 0 AND @totalDiscount > 0 THEN
@employeeId ELSE NULL END,
        @loyaltyDiscountApplied, @bookingId, @employeeId, @tierName
```

```
        );

        -- Update booking status to 'Serviced'
        UPDATE Booking
        SET bookingStatus = 'Serviced'
        WHERE bookingId = @bookingId;

        SET @result = 1;
        SET @message = 'Vehicle return processed successfully. Invoice ' +
@invoiceId + ' generated.';

        COMMIT;
    END TRY
    BEGIN CATCH
        IF @@TRANCOUNT > 0
            ROLLBACK;

        SET @result = 0;
        SET @message = 'Error processing vehicle return: ' + ERROR_MESSAGE();
        SET @invoiceId = NULL;
    END CATCH
END;
GO

PRINT 'Stored procedures created successfully.';
GO
```

Assignment 1

Task 4: Test Cases for Stored Procedures

Group Member Names:

Madhavi Latha Tripuram - C3528532

Hari Krishna Jadda - C3511138

Dhimanshi Piripun Jayasena - C3503338

Lab Timings: **Thursday 1.00pm to 3.00pm**

Course coordinator: **Prof. Mahakim Newton**

Test cases for Stored Procedures

```
-- Test cases For Stored Procedures for GlobalCarHire System
-- INFO6002 Database Management 2 - Assignment 1
-- Group Members:
-- Madhavi Latha Tripuram - C3528532
-- Hari Krishna Jadda - C3511138
-- Dhimanshi Piripun Jayasena - C3503338

-- Test Script for GlobalCarHire Stored Procedures
-- INFO6002 Database Management 2 - Assignment 1

USE GlobalCarHire;
GO

-- First, populate the database with test data
-- (This would normally be in a separate data population script)
-- Here we'll just add minimal data needed for testing
```

```
-- Add a branch

INSERT INTO Branch VALUES ('BR001', 'Sydney CBD', '100 George St, Sydney',
'Australia', '+61291234567', 'AUD', 'Active');


-- Add vehicle categories

INSERT INTO VehicleCategory VALUES ('CAT001', 'Economy', 'Compact cars', 1.00,
15.00);

INSERT INTO VehicleCategory VALUES ('CAT002', 'Luxury', 'Premium vehicles',
2.50, 50.00);


-- Add vehicles

INSERT INTO Vehicle VALUES ('V001', 'Toyota', 'Corolla', 2022, 'ABC123', 'Active',
15000, 5, 'Petrol', 'Automatic', 50.00, 'BR001', 'CAT001');

INSERT INTO Vehicle VALUES ('V002', 'BMW', '520i', 2023, 'XYZ789', 'Active', 5000,
5, 'Petrol', 'Automatic', 120.00, 'BR001', 'CAT002');


-- Add employees

INSERT INTO Employee VALUES ('EMP001', 'John Smith', 'Manager', 'BR001',
'Manager');

INSERT INTO Employee VALUES ('EMP002', 'Sarah Johnson', 'Associate', 'BR001',
'Associate');

INSERT INTO ManagerDetails VALUES ('EMP001', 'Senior', 'All');

INSERT INTO AssociateDetails VALUES ('EMP002', 'Customer Service');


-- Add customers

INSERT INTO Customer VALUES ('CUST001', 'Michael Brown', '0412345678',
'michael@example.com', 'DL123456', 'Gold', 0.10);

INSERT INTO LoyaltyDiscount VALUES ('Silver', 0.05, 5, '5% discount');

INSERT INTO LoyaltyDiscount VALUES ('Gold', 0.10, 10, '10% discount, priority
service');

INSERT INTO LoyaltyDiscount VALUES ('Platinum', 0.15, 20, '15% discount, free
upgrades');

INSERT INTO Customer_LoyaltyDiscount VALUES ('CUST001', 'Gold');
```

```
-- Add services
INSERT INTO Service VALUES ('SER001', 'Self-Drive', 'Standard self-drive rental', 0.00, 'AUD', 'Available', 'Regular');
INSERT INTO Service VALUES ('SER002', 'Chauffeur', 'Professional driver service', 50.00, 'AUD', 'Available', 'Regular');
INSERT INTO RegularServiceDetails VALUES ('SER001', 'Minimum age 21');
INSERT INTO RegularServiceDetails VALUES ('SER002', 'Available only with Luxury vehicles');
INSERT INTO RegularService_VehicleCategory VALUES ('SER001', 'CAT001');
INSERT INTO RegularService_VehicleCategory VALUES ('SER001', 'CAT002');
INSERT INTO RegularService_VehicleCategory VALUES ('SER002', 'CAT002');

-- Add priced services
INSERT INTO PricedService VALUES ('PS001', 50.00, '2023-01-01', '2023-12-31', 'EMP001', 'SER001', 'BR001');
INSERT INTO PricedService VALUES ('PS002', 120.00, '2023-01-01', '2023-12-31', 'EMP001', 'SER001', 'BR001');
INSERT INTO PricedService VALUES ('PS003', 170.00, '2023-01-01', '2023-12-31', 'EMP001', 'SER002', 'BR001');

-- Add extra services
INSERT INTO ExtraService VALUES ('EXT001', 'GPS Navigation', 'Satellite navigation system', 10.00);
INSERT INTO ExtraService VALUES ('EXT002', 'Child Seat', 'Safety seat for children', 15.00);

-- Add drivers
INSERT INTO Driver VALUES ('DRV001', 'Robert Wilson', 'DL987654', '0411223344');

PRINT 'Test data populated successfully.';
GO

-- Test Case 1: Create a valid booking
```

```
DECLARE @result1 INT, @message1 VARCHAR(255);
EXEC sp_CreateBooking
    @bookingId = 'BKG001',
    @modeOfBooking = 'Website',
    @customerId = 'CUST001',
    @serviceStartDate = '2023-06-01',
    @serviceEndDate = '2023-06-05',
    @paymentMethod = 'Credit Card',
    @bookingDeposit = 100.00,
    @employeeId = 'EMP002',
    @pricedServiceIds = 'PS001,PS003',
    @vehicleIds = 'V001,V002',
    @driverIds = 'DRV001',
    @extraServices = 'EXT001:1,EXT002:2',
    @result = @result1 OUTPUT,
    @message = @message1 OUTPUT;

PRINT 'Test Case 1 - Create Booking:';
PRINT 'Result: ' + CAST(@result1 AS VARCHAR) + ', Message: ' + @message1;
PRINT 'Expected: Result = 1, Message = success';
GO

-- Test Case 2: Create booking with invalid vehicle (should fail)
DECLARE @result2 INT, @message2 VARCHAR(255);
EXEC sp_CreateBooking
    @bookingId = 'BKG002',
    @modeOfBooking = 'Website',
    @customerId = 'CUST001',
    @serviceStartDate = '2023-06-01',
    @serviceEndDate = '2023-06-05',
```

```
    @paymentMethod = 'Credit Card',
    @bookingDeposit = 100.00,
    @employeeId = 'EMP002',
    @pricedServiceIds = 'PS001',
    @vehicleIds = 'V999', -- Invalid vehicle
    @driverIds = NULL,
    @extraServices = NULL,
    @result = @result2 OUTPUT,
    @message = @message2 OUTPUT;

PRINT 'Test Case 2 - Create Booking with Invalid Vehicle:';
PRINT 'Result: ' + CAST(@result2 AS VARCHAR) + ', Message: ' + @message2;
PRINT 'Expected: Result = 0, Message = vehicle not available';
GO

-- Test Case 3: Process vehicle return
DECLARE @result3 INT, @message3 VARCHAR(255), @invoiceId VARCHAR(10);
EXEC sp_ProcessVehicleReturn
    @bookingId = 'BKG001',
    @employeeId = 'EMP002',
    @vehicleCondition = 'Minor scratches on rear bumper',
    @mileageReading = 15500,
    @fuelLevel = 75,
    @damagePenalty = 50.00,
    @latePenalty = 0,
    @result = @result3 OUTPUT,
    @message = @message3 OUTPUT,
    @invoiceId = @invoiceId OUTPUT;

PRINT 'Test Case 3 - Process Vehicle Return:';
```

```
PRINT 'Result: ' + CAST(@result3 AS VARCHAR) + ', Message: ' + @message3;
PRINT 'Invoice ID: ' + ISNULL(@invoiceId, 'NULL');
PRINT 'Expected: Result = 1, Message = success, Invoice ID generated';
GO

-- Test Case 4: Process return for non-existent booking (should fail)
DECLARE @result4 INT, @message4 VARCHAR(255), @invoiceId4
VARCHAR(10);
EXEC sp_ProcessVehicleReturn
    @bookingId = 'BKG999',
    @employeeId = 'EMP002',
    @vehicleCondition = 'Good',
    @mileageReading = 10000,
    @fuelLevel = 50,
    @damagePenalty = 0,
    @latePenalty = 0,
    @result = @result4 OUTPUT,
    @message = @message4 OUTPUT,
    @invoiceId = @invoiceId4 OUTPUT;

PRINT 'Test Case 4 - Process Return for Invalid Booking:';
PRINT 'Result: ' + CAST(@result4 AS VARCHAR) + ', Message: ' + @message4;
PRINT 'Expected: Result = 0, Message = booking does not exist';
GO

-- Verify the data after tests
PRINT 'Verifying data after tests:';
SELECT * FROM Booking;
SELECT * FROM Booking_PricedService;
SELECT * FROM Booking_Vehicle;
SELECT * FROM Booking_Driver;
```

```
SELECT * FROM Booking_ExtraService;
SELECT * FROM VehicleTransaction;
SELECT * FROM Invoice;
GO

PRINT 'All test cases completed.';
GO
```